

Week 01 결과 보고서

이름: 강민채 제출일: 0513

문제 1번

■ 문제 내용

정수 n 을 입력받아 1부터 n 까지 중 n 의 약수를 모두 출력하고, 약수의 개수도 함께 출력하라.

■ 소스 코드

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>

int main(void)
{
    int num, total = 0;

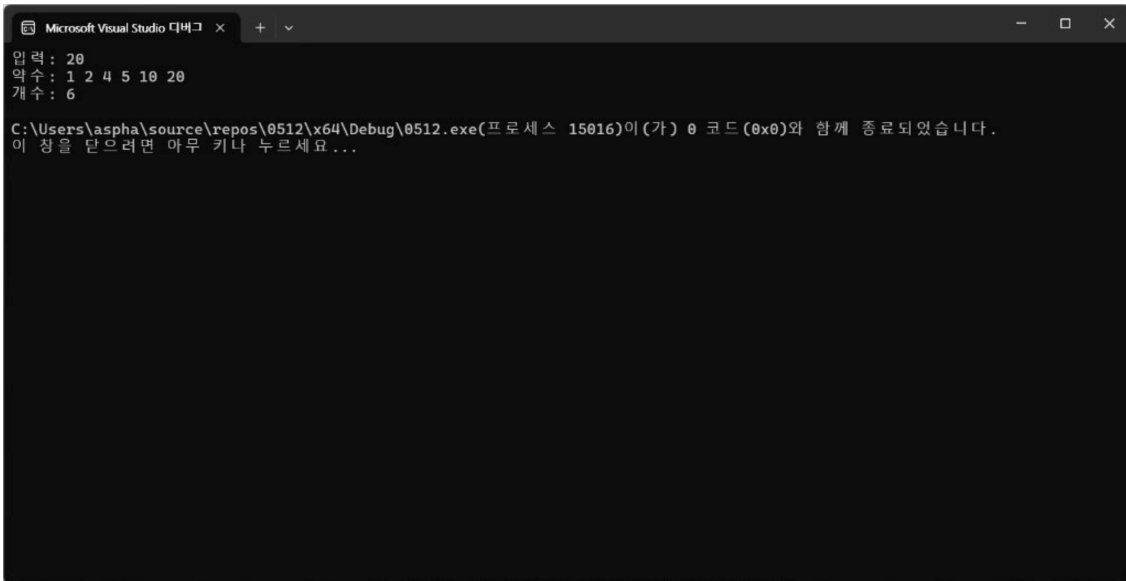
    printf("입력: ");
    scanf("%d", &num);
    printf("약수: ");

    for (int i = 1; i <= num; i++)
    {
        if (num % i == 0)
        {
            printf("%d ", i);
            total++;
        }
    }

    printf("\n");
    printf("개수: %d\n", total);

    return 0;
}
```

■ 실행 화면



```
Microsoft Visual Studio 디버그 콘솔
입력 : 20
약수 : 1 2 4 5 10 20
개수 : 6
C:\Users\aspha\source\repos\0512\x64\Debug\0512.exe(프로세스 15016)이(가) 0 코드(0x0)와 함께 종료되었습니다.
이 창을 닫으려면 아무 키나 누르세요...
```

■ 피드백

어려움을 겪었던 부분:

출력 결과물을 "약수: 1 2 4 5 10 20"과 같이 한 줄에 나열하는 과정

문제를 통해 배우게 된 내용:

for문 사용에 대해 복습하였다.

for문은 정해진 횟수만큼 명령을 반복해서 실행할 때 용이한 반복문이다.

문제 2번

■ 문제 내용

정수 n을 입력받아 소수인지 아닌지 출력하라.

■ 소스 코드

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
```

```
int main(void)
{
    int num, div = 0;

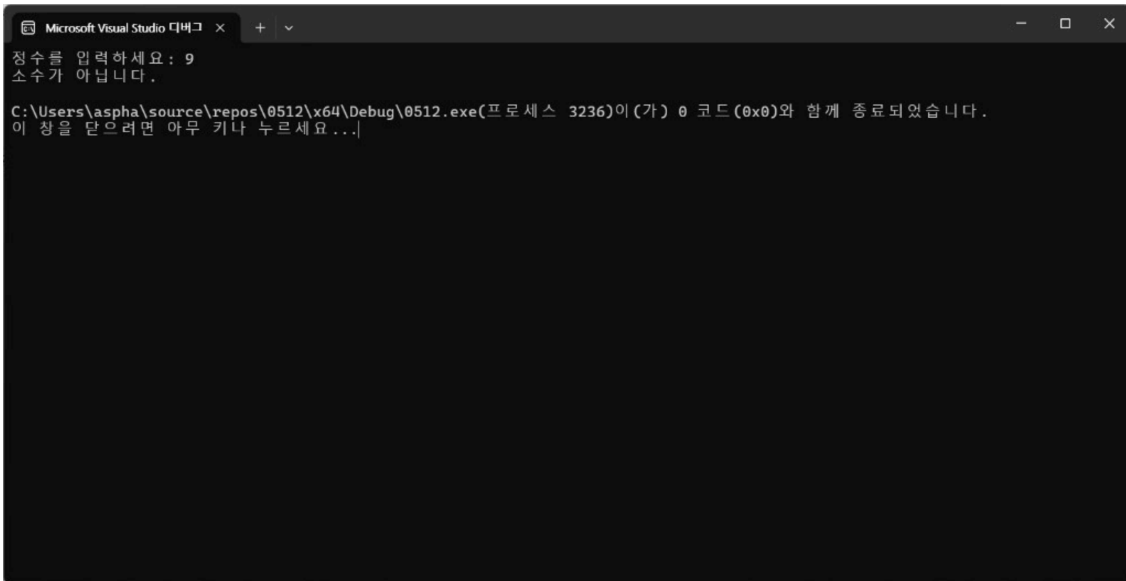
    printf("정수를 입력하세요: ");
    scanf("%d", &num);

    for (int i = 1; i <= num; i++)
    {
        if (num % i == 0)
        {
            div++;
        }
    }

    if (div == 2)
    {
        printf("소수입니다.\n");
    }
    else
    {
        printf("소수가 아닙니다.\n");
    }

    return 0;
}
```

■ 실행 화면



■ 피드백

어려움을 겪었던 부분:

소수를 찾는 과정을 어떻게 설계할 지

문제를 통해 배우게 된 내용:

소수 판별의 핵심 원리를 직접 구현해보며 논리에 따라 코드를 써내리는 과정을 손에 익혔다

문제 3번

■ 문제 내용

정수 n을 입력받아 2부터 n까지의 소수를 모두 출력하고, 개수도 함께 출력하라.

■ 소스 코드

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>

int main(void)
{
    int num, total = 0;

    printf("입력: ");
    scanf("%d", &num);
    printf("소수: ");

    for (int i = 2; i <= num; i++)
    {
        int div = 0;

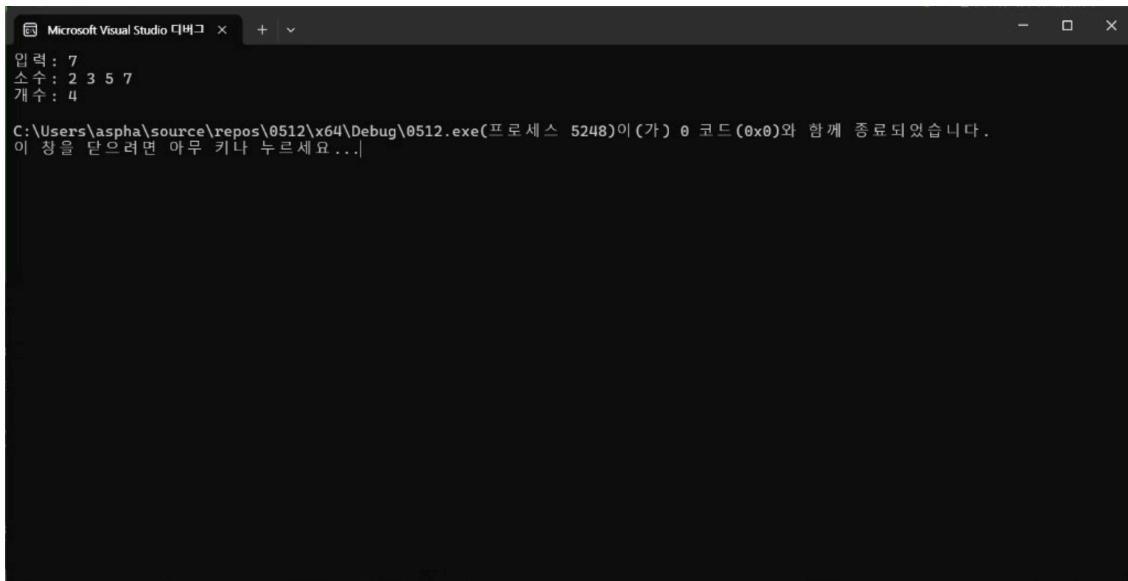
        for (int j = 1; j <= i; j++)
        {
            if (i % j == 0)
            {
                div++;
            }
        }

        if (div == 2)
        {
            printf("%d ", i);
            total++;
        }
    }

    printf("\n");
    printf("개수: %d\n", total);

    return 0;
}
```

■ 실행 화면



```
Microsoft Visual Studio 디버그 x + -
입력 : 7
소수 : 2 3 5 7
개수 : 4
C:\Users\aspha\source\repos\0512\x64\Debug\0512.exe(프로세스 5248)이(가) 0 코드(0x0)와 함께 종료되었습니다.
이 창을 닫으려면 아무 키나 누르세요...
```

■ 피드백

어려움을 겪었던 부분:

for문 중첩 사용

문제를 통해 배우게 된 내용:

for문 중첩의 메커니즘을 직관적으로 파악하는 데 도움이 되었다.

문제 4번

■ 문제 내용

소수를 몇 개 찾을지 입력받아, 그 개수만큼의 소수를 순서대로 출력하라.

■ 소스 코드

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
```

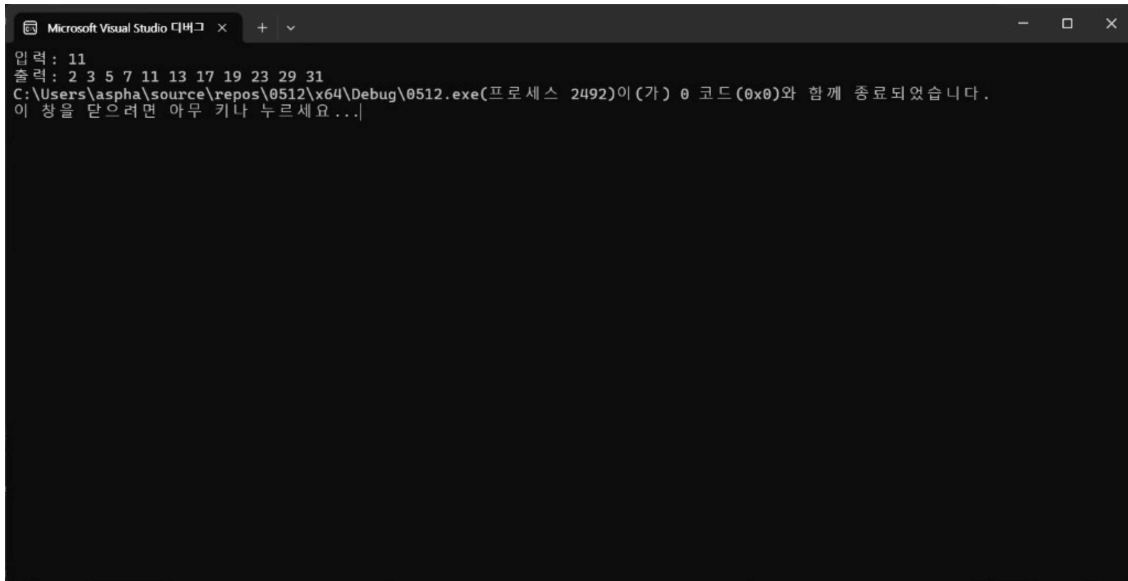
```
int main(void)
{
    int target, num = 2, count = 0;

    printf("입력: ");
    scanf("%d", &target);
    printf("출력: ");

    while (count < target)
    {
        int div = 0;

        for (int i = 1; i <= num; i++)
        {
            if (num % i == 0)
            {
                div++;
            }
        }
        if (div == 2)
        {
            printf("%d ", num);
            count++;
        }
        num++;
    }
    return 0;
}
```

■ 실행 화면



```
Microsoft Visual Studio 디버그 x + v
입력 : 11
출력 : 2 3 5 7 11 13 17 19 23 29 31
C:\Users\aspha\source\repos\0512\x64\Debug\0512.exe(프로세스 2492)이(가) 0 코드(0x0)와 함께 종료되었습니다.
이 창을 닫으려면 아무 키나 누르세요 ...|
```

■ 피드백

어려움을 겪었던 부분:

while문 안으로 변수 div를 초기화하지 않았을 때, 소수가 모두 출력되지 않았다.

문제를 통해 배우게 된 내용:

중첩 루프 구조에서 변수의 선언 및 초기화 위치가 프로그램 전체의 논리에 미치는 영향을 학습하였다.

1번 문제

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n;
```

```
    printf("정수를 입력하세요: ");
```

```
    scanf_s("%d", &n);
```

```
    for (int i = n; i >= 1; i--) //i를 감소하며 입력받은 정수의 만큼 열 생성 ex)5입력 -> 5 4 3 2 1
```

```
    {
```

```
        for (int j = 1; j <= i; j++) //j가 i보다 커질 때 까지 반복 이후 줄바꿈 명령어로 이동
```

```
        {
```

```
            printf("*");
```

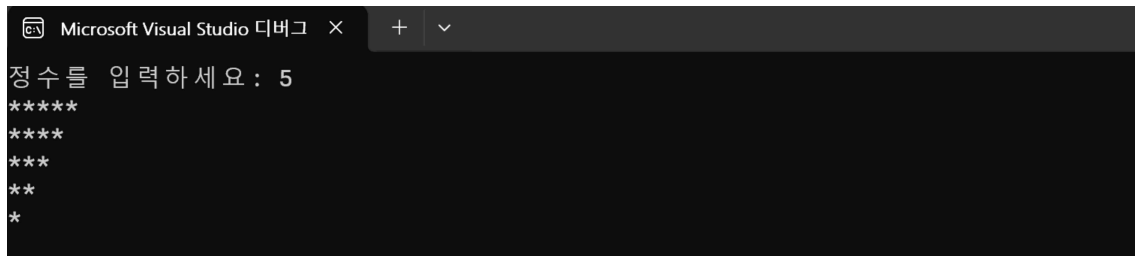
```
        }
```

```
        printf("\n");//줄바꿈 이후 다시 바깥 for문 조건이 거짓이 될 때 까지 반복
```

```
    }
```

```
    return 0;
```

```
} // for문의 조건식이 너무 어려웠다 해당 조건이 참이 될 때 까지가 아니라 참인 동안 반복임을 자꾸 까먹었다.
```



```
Microsoft Visual Studio 디버그 × + v
정수를 입력하세요 : 5
*****
*****
****
***
**
*
```

코드의 조건식과 참, 거짓을 0과 1(on/off)와 비교해서 기억해놓고 익숙해지도록 하면 좋을것 같습니다

2번 문제

```
#include <stdio.h>
```

```
int main() {
```

```
    int n;
```

```
    printf("정수를 입력하세요: ");
```

```
    scanf_s("%d", &n);
```

```
    for (int i = 0; i < n; i++) //i는 사각형의 높이, 입력한 정수와 i값이 같아지면 종료 ex) 5입력 -> 0 1 2 3 4
```

```
    {
```

```
        for (int j = 0; j < n; j++)
```

```
//j는 사각형의 밑변의 길이, 입력한 정수와 j값이 같아지면 종료 ex) 4입력 -> 0 1 2 3
```

```
    {
```

```
        if (i == 0 || i == n - 1 || j == 0 || j == n - 1) //만약 네 가지 수식중 하나라도 해당하면
```

```
        {
```

```
            printf("*"); //별 출력
```

```
        }
```

```
    else
```

```
    {
```

```
        printf(" "); // 그렇지 않으면 공백 출력으로 사각형의 면 만들기
```

```
    }
```

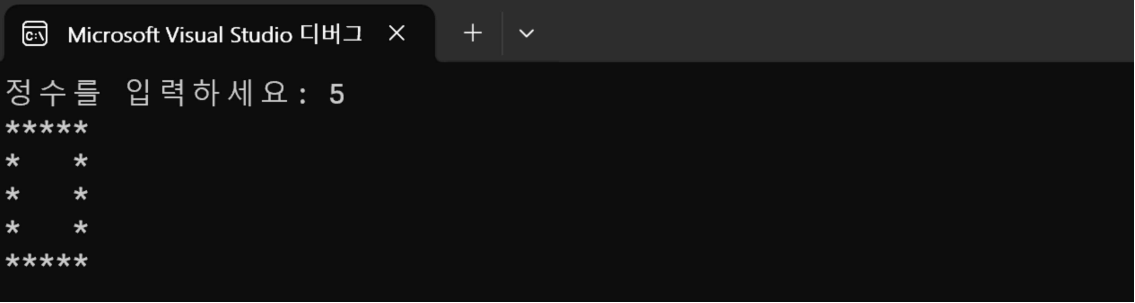
```
    }
```

```
    printf("\n"); // 가로열을 n-1만큼 출력하면 줄바꿈
```

```
}
```

```
    return 0;
```

```
}
```



```
Microsoft Visual Studio 디버그 × + ▾
정수를 입력하세요 : 5
*****
*   *
*   *
*   *
*****
```

// 1번 문제에서 막혔던 부분을 풀어내고 원리를 이해 하고나니 쉬울거라고 생각 했으나 if문을 사용해야 하는지
// 갈피를 잡지 못 하여서 조금 걸렸었다. if문을 사용해야 한다는 말을 전해듣고 나서는 교재에서 나온 if문 예제를
// 참고하여서 천천히 틀을 만들어 완성하였다.
// 문제의 크기를 여러 부분(높이, 밑변)으로 잘 쪼개서 해결한 점이 좋은 것 같다.

3번

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
int main(void) {
    int n, i, j;
    printf("n=");
    scanf("%d", &n);
    for (i = 1; i < n + 1; i++) {
        /*
            for (j = 0; j < n-i; j++) {
                // 공백을 출력해야 하는 횟수 계산 = n(출력할 공간의 크기) - i(별의 갯수)
                printf(" ");
            }
        */
        for (j = 0; j < i; j++) {
            printf("*");
        }
        printf("\n");
    }
    return 0;
}
```

// 교재에 나온 그대로 작성해봤는데 공백을 먼저 쓰는 부분에서 너무 막혀서 완성을 못 했음

// 컴퓨터가 출력할하는 범위가 어떻게 되는지 생각해보고 사용자의 눈에 보이는 부분과 안 보이지만 컴퓨터의

입장에서는 존재하는(보이는) 부분을 인지하여 컴퓨터의 입장에서 코드를 작성하는 법을 기르면 좋을 것 같습니다.

Microsoft Visual Studio 디버그 × + ▾

```
n=5
  *
 **
***
****
*****
```

4번

```
#include <stdio.h>
```

```
int main() {
```

```
    int n1, n2;
```

```
    int max, min;
```

```
    printf("정수를 입력하세요: ");
```

```
    scanf_s("%d %d", &n1, &n2);
```

```
    max = (n1 > n2) ? n1 : n2;
```

```
    min = (n1 < n2) ? n1 : n2;
```

```
    for (int i = 0; i < min; i++)
```

//i는 사각형의 높이, 입력한 정수와 i값이 같아지면 종료 ex) 5입력 -> 0 1 2 3 4

```
    {
```

```
        for (int j = 0; j < max; j++)
```

//j는 사각형의 밑변의 길이, 입력한 정수와 j값이 같아지면 종료 ex) 4입력 -> 0 1 2 3

```
        {
```

```
            if (i == 0 || i == min - 1 || j == 0 || j == max - 1)
```

//만약 네 가지 수식중 하나라도 해당하면

```
            {
```

```
                printf("*"); //별 출력
```

```
            }
```

```
        else
```

```
        {
```

```
            printf(" "); // 그렇지 않으면 공백 출력으로 사각형의 면 만들기
```

```
        }
```

```

    }

    printf("\n"); // 가로열을 n-1만큼 출력하면 줄바꿈
}

return 0;
}

// 삼항 연산자를 통해서 두 정수의 대수 비교를 통해 큰 수를 max, 작은 수를 min변수에
// 저장하여 2번 문제와 같이

// max수 만큼 가로열을, min의 수 만큼 세로열을 가진 직사각형을 만들었다.

// 문제를 작은 부분(가로, 세로) 로 쪼개서 처리한 점이 좋음

```



```

Microsoft Visual Studio 디버그
+
정수들 입력하세요 : 5 3
*****
*      *
*****

```

Week 01 결과 보고서

이름 : 오병욱

제출일 : 05/12

문제 1번

■ 문제 내용

1번

정수 n 을 입력받아 높이 n 인 역삼각형을 출력하라.

```
입력: 4
****
***
**
*

```



힌트: i 가 0부터 시작하면 i 번째 줄에 $(n-i)$ 개의 별 출력. 안쪽 for의 반복 횟수를 i 에 따라 바꾸는 것이 핵심

■ 소스코드

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int n;
```

```
    printf("정수 n 입력: ");
```

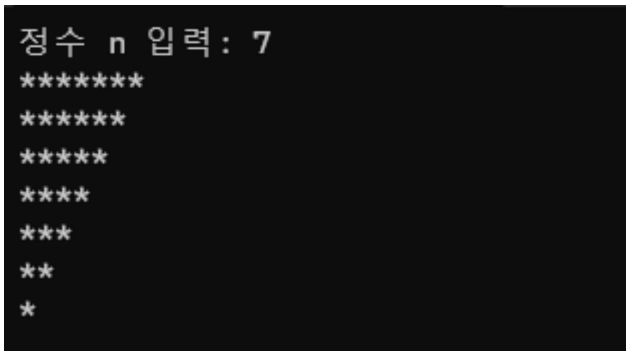
```
    scanf("%d", &n);
```

```

for (int i = 0; i < n; i++)
{
    for (int j = i; j < n; j++)
        printf("*")
    printf("\n");
}
}

```

■ 실행화면



```

정수 n 입력: 7
*****
*****
*****
****
***
**
*

```

■ 피드백

어려움을 겪었던 부분

줄이 내려갈수록 별의 개수가 하나씩 줄어들도록 설정하는 부분에서 많이 헛갈렸다.

문제를 통해 배우게 된 내용

반복 횟수를 $(n-i)$ 처럼 계산하여 출력 모양을 자유롭게 만들 수 있다는 것을 배우게 되었다.

문제 2번

■ 문제 내용

2번

정수 n 을 입력받아 $n \times n$ 테두리만 있는 사각형을 출력하라.

입력: 4

* *
* *

힌트: 행 번호로 첫 줄-마지막 줄 구분 ($i == 0 \parallel i == n-1$), 나머지 줄은 열 번호로 양쪽 끝 구분 ($j == 0 \parallel j == n-1$)

■ 소스코드

```
#include <stdio.h>

int main(void)
{
    int n;

    printf("정수 입력: ");
    scanf("%d", &n);

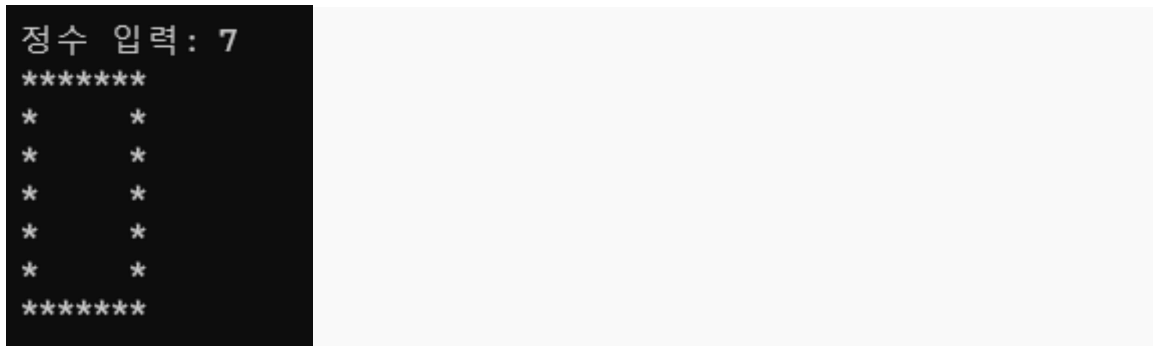
    for (int i = 0; i < n; i++)
    {
        printf("*");
        for (int j = 0; j < n-2; j++)
        {
            if (i == 0 || i == n - 1)
                printf("*");

            else
                printf(" ");
        }

        printf("*");

        printf("\n");
    }
}
```

■ 실행화면



■ 피드백

어려움을 겪었던 부분

어려웠던 부분은 첫 줄과 마지막 줄, 그리고 중간 줄을 각각 다르게 출력해야 한다는 점이었다.

특히 가운데 부분은 양쪽 끝만 *을 출력하고 나머지는 공백을 출력해야 해서 조건문을 어떻게 작성할지 헷갈렸다.

문제를 통해 배우게 된 내용

행과 열의 위치에 따라 다른 값을 출력하는 원리를 이해하게 되었고, 공백 출력도 모양을 만드는 데 중요하다는 것을 알게 되었다.

또한 반복문을 활용하면 다양한 도형을 만들 수 있다는 점이 흥미로웠다.

문제 3번

■ 문제 내용

3번

정수 n 을 입력받아 높이 n 인 오른쪽 정렬 직각삼각형을 출력하라.

```
입력: 4
*
**
***
****
```

힌트: i 번째 줄에 공백 $(n-i)$ 개 먼저 출력한 뒤 별 i 개 출력. 공백용 `for`와 별용 `for`를 따로 두기

■ 소스코드

```
#include <stdio.h>

int main(void)
{
    int n;

    printf("정수 n 입력: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
    {
        for (int j = i; j < n-1; j++)
            printf(" ");

        for (int j = 0; j < i+1; j++)
            printf("*");

        printf("\n");
    }
}
```

■ 실행화면

```
정수 n 입력: 5
*
**
***
****
*****
```

■ 피드백

어려움을 겪었던 부분

문제 1번과 다르게 별만 출력하는 것이 아니라 공백까지 함께 계산해야 한다는 점이 어려웠다.

문제를 통해 배우게 된 내용

반복문의 반복 횟수를 조절하면 다양한 정렬 형태를 구현할 수 있다는 점을 이해하게 되었다.

문제 4번

■ 문제 내용

4번

두 정수를 입력받아 큰 수를 가로·세로, 작은 수를 안쪽 빈 공간의 한 변으로 하는 속이 빈 직사각형을 출력하라.

입력: 5 3

* *
* *
* *

힌트: 두께 = (바깥 - 안쪽) / 2 를 먼저 구하기. 행/열 번호가 두께 미만이거나 바깥-두께 이상이면 *, 나머지는 공백

■ 소스코드

```
#include <stdio.h>

int main(void)
{
    int n1, n2, t, b, s;

    printf("두 정수 입력: ");
    scanf("%d %d", &n1, &n2);

    if (n1 > n2)
        b = n1, s = n2;
    else
        b = n2, s = n1;

    t = (b - s) / 2;

    for (int i = 0; i < b; i++)
    {
        for (int j = 0; j < b; j++)
        {
            if (i < t || i >= b - t || j < t || j >= b - t)
                printf("*");
            else
                printf(" ");
        }

        printf("\n");
    }
}
```

■ 실행화면

```
두 정수 입력: 5 7
*****
*       *
*       *
*       *
*       *
*       *
*       *
*****
```

■ 피드백

어려움을 겪었던 부분

두께를 (바깥 - 안쪽) / 2로 계산하는 부분을 이해하는 데 시간이 걸렸다.

문제를 통해 배우게 된 내용

행과 열의 번호를 이용해 출력 범위를 제어하는 방법을 이해하게 되었고, 여러 조건을 조합하면 다양한 형태를 만들 수 있다는 것을 알게 되었다.

또한 문제의 규칙을 먼저 분석한 뒤 코드를 작성하는 것이 중요하다는 점도 배울 수 있었다.